

Morphic Studio

Document 35 — Master Development Playbook

Document ID: MS-035

Version: 1.0

Status: Living Development Guide

Priority: Highest

Purpose

The Master Development Playbook transforms the Morphic Studio Blueprint (Documents MS-000 through MS-034) into an actionable implementation strategy.

While the previous documents describe **what** Morphic Studio is and **why** it exists, this playbook defines **how it will be built, in what order, with which tools, and according to which engineering principles.**

It serves as the primary operational guide for day-to-day development.

Vision

Development should be:

- Structured
- Repeatable
- Documented
- Modular
- Beginner-friendly
- Professional
- Scalable

The goal is not simply to finish the project, but to build it in a way that remains understandable, maintainable, and extensible for years to come.

Development Philosophy

Every feature should follow the same lifecycle:

Idea



Specification



Architecture Review



UI/UX Design



Implementation



Testing



Documentation



Deployment



Review



Iteration

Skipping steps creates technical debt.

Guiding Principles

1. Build one complete feature at a time.
 2. Avoid rewriting systems unnecessarily.
 3. Prefer modular components over tightly coupled code.
 4. Reuse assets and logic wherever possible.
 5. Document major decisions.
 6. Test before expanding.
 7. Keep the architecture clean.
 8. Optimize only after correctness.
-

Primary Toolchain

Development

- Replit

Primary coding environment.

Responsible for:

- Frontend
 - Backend
 - Database integration
 - APIs
 - Authentication
 - Testing
 - Deployment
-

UI / UX

Google Stitch

Responsible for:

- Wireframes
- High-fidelity UI concepts
- Component exploration
- Design iteration

Approved designs are implemented inside Replit rather than treated as production code.

Source Control

Git + GitHub

Every meaningful change should be committed with descriptive messages.

Suggested workflow:

Feature Branch

↓

Development

↓

Testing

↓

Merge

↓

Release

AI Engineering Assistants

ChatGPT

Best used for:

- System architecture
- Debugging
- API design
- Documentation
- Code reviews
- Refactoring
- Learning unfamiliar concepts

Gemini

Best used for:

- Design critique
- Product ideation
- Alternative implementation approaches
- Long-form reasoning
- Secondary reviews

Treat AI as a collaborator, not an unquestioned authority.

Recommended Open-Source Stack

Frontend:

- React
- TypeScript
- Tailwind CSS

Backend:

- Node.js
- Express (or NestJS later if needed)

Database:

- PostgreSQL

ORM:

- Prisma

Authentication:

- Better Auth (or another actively maintained open-source solution)

AI Orchestration:

- LangGraph (if workflow complexity grows)
- Custom orchestration where appropriate

Image Generation:

- ComfyUI

Animation Pipeline:

- Blender (automation later)

- FFmpeg
- OpenToonz (optional for specific workflows)

Voice:

- Kokoro TTS (preferred)
- XTTS v2 (optional)

Storage:

- S3-compatible object storage (future)
 - Local/Replit storage for early development
-

Development Phases

Phase 1 — Foundation

Goals:

- Initialize repository.
- Configure Replit.
- Configure database.
- Authentication.
- Dashboard shell.

Exit Criteria:

- Users can sign in.
 - Project structure exists.
 - CI-ready repository.
-

Phase 2 — Story Systems

Build:

- Story Bible
- Creative Memory Engine
- Character Manager
- World Manager

Exit Criteria:

- Projects can store structured story data.
-

Phase 3 — Asset Systems

Build:

- Asset Library
- Uploads
- Metadata
- Versioning
- Search

Exit Criteria:

- Assets are reusable and searchable.
-

Phase 4 — AI Integration

Integrate:

- Story analysis
- Character generation
- Background generation
- Asset reuse logic

Exit Criteria:

- AI can analyze a script and populate the Story Bible with reusable assets.
-

Phase 5 — Comic Studio

Build:

- Storyboard
- Panel editor
- Comic generator
- Export

Exit Criteria:

- End-to-end comic workflow is functional.
-

Phase 6 — Animation Studio

Build:

- Timeline
- Motion
- Camera
- Audio
- Export

Exit Criteria:

- Motion comics and basic animations are supported.
-

Phase 7 — Platform Growth

Add:

- Collaboration
- Templates
- Marketplace
- Plugins
- Public APIs

Exit Criteria:

- Platform supports external creators and extensions.
-

Weekly Development Cycle

Every week should include:

1. Planning
2. Implementation
3. Testing
4. Documentation
5. Review

Avoid spending weeks coding without updating documentation.

Daily Workflow

Suggested routine:

Morning:

- Review current milestone.
- Read relevant blueprint documents.
- Define today's objective.

Development:

- Build one focused feature.
- Commit regularly.
- Keep changes small and understandable.

End of Day:

- Test changes.
 - Update documentation.
 - Record lessons learned.
 - Plan the next session.
-

Definition of Done

A feature is complete only when:

- It functions correctly.
 - It has been tested.
 - Documentation is updated.
 - Related blueprint documents remain accurate.
 - Code is committed.
 - No critical known defects remain.
-

Learning Roadmap

As you build, prioritize learning:

1. Git & GitHub
2. HTML & CSS
3. JavaScript
4. TypeScript

5. React
6. Node.js
7. PostgreSQL
8. APIs
9. Authentication
10. AI integration
11. Performance optimization
12. Deployment

Each skill should be learned in the context of Morphic Studio rather than in isolation.

Coding Standards

Follow these principles:

- Write readable code before clever code.
 - Use descriptive names.
 - Keep functions focused.
 - Separate concerns.
 - Avoid unnecessary duplication.
 - Add comments only when they explain intent, not obvious syntax.
 - Refactor when patterns become repetitive.
-

Documentation Standards

Every major feature should include:

- Purpose
- Architecture
- Data flow
- API interactions
- User flow
- Future improvements

Documentation should evolve with the codebase.

Decision Framework

Before adding any feature, ask:

1. Does it solve a real creator problem?

2. Does it align with Morphic Studio's philosophy?
3. Can it be built modularly?
4. Will it remain useful in future versions?
5. Does it introduce unnecessary complexity?

If several answers are "no," reconsider the feature.

Milestone Reviews

At the end of every milestone, review:

- What was built?
- What was learned?
- What should be improved?
- What technical debt was introduced?
- Which documents need updating?

Continuous reflection prevents long-term architectural drift.

Success Metrics

Track progress through:

- Stable releases
- Features completed
- Documentation coverage
- Test pass rate
- Personal learning milestones
- Creator feedback
- System reliability

Success is measured by steady, sustainable progress—not by rushing features.

Final Principle

Morphic Studio is not just a software project—it is a long-term creative platform. Every decision should make the platform easier to understand, easier to maintain, and more valuable to creators. Build patiently, document thoroughly, and let the architecture evolve through deliberate, well-tested improvements rather than shortcuts.

End of Document 35